

Environment Setup Guide

Environment Setup Guide | NOVA IMS (2026)

Welcome to the **Programming Methods** coursework repository. This course focuses on building scalable, modular, and performance-aware Python data pipelines.

To ensure reproducibility this project strictly uses `uv` for dependency and Python version management, alongside a local Large Language Model (LLM) engine for lab assignments.

Required System Tools

Please follow the official provider links below to install the necessary software for your operating system.

Tool	Version	Purpose	Official Installation Guide
Git	v2.45+	Version control engine (required by VS Code for Git operations).	Download Git
VS Code	v1.91+	Primary IDE for code, notebooks, and Git integration.	Download VS Code
Ollama	v0.3.0+	Local LLM inference server (runs open-weight models natively).	Download Ollama
uv	v0.2.20+	Python and package manager.	Install uv (Astral)

VS Code Extensions

To support Python development and render Jupyter Notebooks properly in VS Code, open your terminal and run the following command:

macOS Note: If your terminal says `command not found: code`, you need to enable the VS Code CLI first. Open VS Code, press `Cmd + Shift + P`, type **Shell Command: Install 'code' command in PATH**, select it, and then restart your terminal.

```
code --install-extension ms-python.python \
--install-extension ms-python.vscode-pylance \
--install-extension ms-toolsai.jupyter \
--install-extension ms-toolsai.jupyter-keymap \
--install-extension ms-toolsai.jupyter-renderers \
```

Project Initialization & Virtual Environment

This project uses a `pyproject.toml` file to manage all Python dependencies. With `uv`, you do not need to manually create a virtual environment or run `pip install`—it handles everything in one step.

Open your terminal inside the root folder of this repository and run the following commands:

```
# 1. Pin the Python version for this specific folder
uv python pin 3.12

# 2. Automatically create the .venv and install dependencies
uv sync

# 3. Activate the virtual environment
# -> Run this on macOS / Linux:
source .venv/bin/activate

# -> Run this on Windows:
.venv\Scripts\activate
```

VS Code Tip: Selecting the Right Kernel

When you open a Python file (`.py`) or a Jupyter Notebook (`.ipynb`) in VS Code for the first time, you need to tell VS Code to use your new virtual environment.

1. Click the **Python version / Select Kernel** button in the top right corner (for notebooks) or bottom right corner (for scripts).
2. Click **Select Another Kernel** (if prompted), then choose **Python Environments**.
3. Choose the interpreter located inside your project folder:
 - **Mac/Linux:** `./.venv/bin/python`
 - **Windows:** `.\.venv\Scripts\python.exe`

Local LLM Setup (Ollama)

For our API and Pydantic pipeline labs, you will need to download a lightweight local model. Open your terminal and run **one** of the following commands based on your machine's hardware:

```
# Option A: Recommended Default (Requires ~2GB RAM)
# Best balance of speed and JSON-following capabilities.
ollama pull llama3.2:1b

# Option B: For Older Laptops (Requires < 1GB RAM)
# The absolute smallest model. Choose this if your laptop struggles with RAM.
ollama pull qwen2.5:0.5b

# Option C: For Coding Tasks
# A lightweight alternative fine-tuned specifically for Python coding tasks.
ollama pull qwen2.5-coder:1.5b
```

To test your model directly in the terminal, use the `run` command followed by the model name:

```
# Change "llama3.2:1b" if you downloaded Option B or C instead
ollama run llama3.2:1b
```

Terminal Tip

Once you are inside the chat, you can type `/bye` or press `Ctrl + D` at any time to exit the model and return to your normal terminal.